



Отчёт по проверке ИИ-контента

Сводная информация

Автор: Не указано

Название документа: Пояснительная диплома Лямов

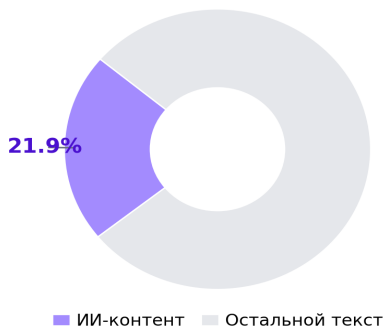
Тип документа: Дипломная работа

Подразделение: Не указано

Проверяющий: Нурбол Лямов (nurbollamov@gmail.com)

Дата проверки: 16.05.2026 12:54

Результаты проверки ИИ-контента



ИИ-контент: **21.9%**

Показатель отражает долю текста, которую система классифицировала как сгенерированную моделями искусственного интеллекта (например, ChatGPT). Значение носит вероятностный характер и не является прямым доказательством нарушения академической честности.

В данном отчёте намеренно не отображаются показатели оригинальности, совпадений, цитирований, самоцитирований и подозрительной активности (обходы, замены символов и т.п.). Полная информация доступна в основном отчёте антиплагиата.

Система анализа ИИ-контента использует лингвистические и статистические признаки текста. Результаты должны интерпретироваться проверяющим с учётом содержания работы, уровня автора, требований организации и контекста подготовки документа.

Текст документа (с выделением ИИ-фрагментов)

Введение На сегодняшний день даже в небольшой локальной сети постоянно идёт фоновая подозрительная активность попытки — сканирования — портов, нестандартные подключения, резкие всплески трафика. Часть таких событий не блокируется автоматически и нередко остаётся незамеченной вплоть до момента, когда уже возникают реальные проблемы с работой сервисов и сетевой инфраструктуры. Именно поэтому для администратора сети особенно важна система, которая не просто фильтрует трафик, а целенаправленно фиксирует и наглядно показывает признаки возможной атаки, позволяя своевременно на неё отреагировать [1]. В данной работе будет рассматриваться разработка прототипа IDS для локальной сети. Выбран практический подход: создать рабочее приложение, которое можно запустить в учебной среде, проверить на тестовых сценариях и в дальнейшем использовать как основу для продолжения развития системы [2]. Цель проектной работы — создать прототип системы обнаружения вторжений, который в реальном времени анализирует сетевой трафик, выявляет подозрительные события и сохраняет результаты для последующего детального разбора. Для достижения поставленной цели в проекте будут решены следующие задачи: - проанализировать предметную область и существующие решения IDS; - сформулировать требования к разрабатываемому прототипу; - выбрать технологии разработки и обосновать их применение; - спроектировать структуру системы и взаимодействие её модулей; - реализовать захват трафика, модуль детекции, журналирование, хранение данных и графический интерфейс; - провести тестирование и оценить работоспособность полученного решения. Объект исследования — процессы сетевого обмена данными в локальной сети в контексте обеспечения информационной безопасности. Предмет исследования — методы программного обнаружения сетевых атак и аномалий на основе анализа сетевого трафика. Практическим результатом работы станет прототип IDS HunterEye, который будет включать базовые механизмы захвата пакетов, набор правил обнаружения, хранение событий и пользовательский интерфейс для мониторинга. Полученная система не будет претендовать на промышленный уровень, однако в полной мере закроет ключевые учебные и демонстрационные задачи: продемонстрирует полный цикл от получения сетевых данных до регистрации инцидента и вывода информации оператору. В дальнейших разделах последовательно рассмотрены предпроектные исследования, выбор технологий, проектирование системы, реализация программных модулей и результаты тестирования.

1 Предпроектные исследования Этап предварительных исследований выполняет сразу несколько функций: уточняет границы будущей разработки, связывает теоретическую постановку задачи с практикой эксплуатации локальной сети и формирует набор требований, которые затем переходят в техническую реализацию. Для темы, связанной с системой обнаружения вторжений, этот этап особенно важен, поскольку эффективность конечного решения напрямую зависит от того, насколько корректно определены типовые угрозы, наблюдаемые признаки атак и условия работы программного прототипа [3]. В данной работе предпроектные исследования ориентированы на создание прикладного решения, которое может быть развернуто в учебной или малой корпоративной локальной сети без сложной внешней инфраструктуры. Это означает, что на раннем этапе необходимо учитывать не только классические вопросы информационной безопасности, но и ограничения среды: доступность вычислительных ресурсов, необходимость работы в операционной системе Windows, зависимость от драйвера захвата пакетов, а также требования к наглядности результатов для пользователя [4]. Ключевая идея предпроектного этапа состоит в том, что разрабатываемая IDS должна быть не абстрактной «теоретической моделью», а инструментом, способным в режиме, близком к реальному времени: — принимать сетевой трафик с выбранного интерфейса; — выделять признаки потенциально опасной активности; — формировать и сохранять события безопасности; — предоставлять оператору понятный интерфейс контроля и анализа. На этом основании в рамках главы решаются три взаимосвязанные задачи. Первая — анализ предметной области: определение места IDS в контуре защиты локальной сети, классификация типовых угроз и выявление признаков, пригодных для программного

обнаружения. Вторая — изучение существующих разработок (аналогов), чтобы определить, какие подходы уже доказали практическую состоятельность и какие решения целесообразно адаптировать в прототипе. Третья — формализация постановки задачи в терминах будущей реализации: цель, функциональные и нефункциональные требования, ожидаемые результаты. Результатом предпроектных исследований является обоснованная база для следующих разделов проекта. Именно на этом этапе определяется, что прототип должен реализовывать полный минимально необходимый цикл IDS: захват трафика, анализ, детекцию, журналирование, хранение и визуализацию. При этом в работу сознательно закладывается принцип поэтапного развития: сначала реализуются устойчивые и проверяемые методы обнаружения (пороговые и сигнатурные правила), а затем формируются направления для дальнейшего усложнения системы (расширение правил, интеграция с внешними источниками телеметрии, интеллектуальные методы анализа) [5].

7 Таким образом, предпроектные исследования выступают фундаментом всей работы: они задают логику проектирования, снижают риски разработки и обеспечивают связь между требованиями информационной безопасности и конкретными программными модулями создаваемого прототипа IDS.

1.1 Анализ предметной области

Предметная область проектной работы охватывает процессы мониторинга и анализа сетевого трафика в локальной сети с целью выявления признаков вторжений и аномальной активности. В современных условиях локальная сеть, даже без прямой публичной экспозиции, остаётся уязвимой для множества сценариев: внутренние — инциденты, активность зараженного — узла, несанкционированный доступ из гостевого сегмента, сканирование сервисов и попытки перегрузки сетевых служб [22]. Локальная сеть как объект защиты характеризуется высокой динамикой: состав активных устройств и интенсивность трафика постоянно меняются, а сервисы работают в различных режимах нагрузки. Поэтому администратору важны не только средства блокирования трафика, но и средства наблюдения, позволяющие своевременно выявлять отклонения от нормального поведения. Эту функцию в контуре информационной безопасности выполняют системы обнаружения вторжений. Содержательно IDS решает задачу распознавания опасной активности по наблюдаемым признакам. В практических условиях обнаружение обычно строится на комбинации подходов: 1. сигнатурный подход — поиск известных шаблонов, соответствующих ранее описанным сценариям атак; 2. пороговый подход — фиксация превышения заранее заданных количественных характеристик (например, числа пакетов за интервал времени); 3. поведенческий (аномальный) подход — выявление отклонений от типичного сетевого профиля [5]. Для учебного прототипа, разрабатываемого в данной работе, наиболее рациональной является комбинация первых двух подходов с элементами поведенческой оценки. Такой выбор объясняется несколькими причинами. Во-первых, пороговые и сигнатурные правила легко интерпретируются и позволяют прозрачно объяснить каждое срабатывание. Во-вторых, они относительно просты в реализации и тестировании. В-третьих, такой подход дает достаточную практическую ценность для демонстрации полного цикла работы IDS в ограниченной среде. В рассматриваемой предметной области особый интерес представляют следующие типы угроз: — сканирование портов, указывающее на разведку сетевых сервисов; — резкий рост числа пакетов от одного источника (признаки DoS-подобной активности); — аномальная SYN-активность как индикатор возможного SYN flood; — обращения к заведомо подозрительным портам и сервисам; — подозрительная ACP-активность в локальном сегменте [6]. Каждый из перечисленных сценариев может быть описан через измеримые сетевые признаки: количество пакетов за окно наблюдения, число уникальных портов назначения, состав TCP-флагов, параметры адресации и контекст времени. Это важно, потому что предметная область диплома предполагает именно программно реализуемый контроль, а значит, любые утверждения о «подозрительности» должны быть формализованы в виде проверяемых правил. Отдельного внимания требует вопрос качества детекции. В реальной сети невозможно полностью — устранить ложные — срабатывания, поскольку интенсивность трафика может меняться по легитимным причинам. Предметная

область диктует необходимость компромисса между чувствительностью и устойчивостью системы: правила должны фиксировать реальные инциденты, но не перегружать оператора избыточными предупреждениями. Именно поэтому в проекте предусматривается настройка порогов и временных окон, а также централизованное управление правилами [7]. С точки зрения архитектуры безопасности IDS не заменяет межсетевой экран или антивирус, а дополняет их. Если межсетевой экран реализует политику допуска/запрета, то IDS формирует аналитический уровень наблюдения: «что происходило», «когда», «с каким уровнем критичности», «какой источник вероятно участвовал в инциденте». Для локальной сети это особенно полезно, поскольку многие инциденты связаны с внутренним трафиком, который классические периметровые средства могут не интерпретировать как угрозу. При анализе предметной области необходимо учитывать технологические ограничения выбранной платформы. Для ОС Windows захват пакетов требует наличия специализированного драйвера (Npcap или аналог), а полнота наблюдаемой картины трафика зависит от точки установки системы и параметров сетевого интерфейса [8]. Прототип ориентирован на мониторинг трафика, доступного конкретному интерфейсу, и не претендует на уровень распределённой корпоративной системы класса SOC. Для целей проекта такой масштаб обоснован: он позволяет построить корректный и воспроизводимый инструмент мониторинга в пределах локального контура [9]. С прикладной точки зрения предметная область формирует набор обязательных функций будущей системы: — непрерывный сбор сетевых данных; — преобразование пакетов к внутреннему формату для анализа; — применение набора правил обнаружения; — генерация событий с указанием типа, времени и критичности; — хранение результатов в журнале и базе данных; — визуальное отображение текущего состояния и истории событий. Таким образом, анализ предметной области показывает, что задача разработки прототипа IDS для локальной сети является актуальной и технически реализуемой в рамках проектной работы. Формализуемость основных сценариев атак делает возможным создание модульной системы с проверяемыми результатами, что служит основанием для последующей постановки задачи в формате технических требований к проектируемому прототипу.

1.2 Анализ существующих разработок

Для обоснования архитектуры разрабатываемого прототипа IDS необходимо рассмотреть существующие решения, применяемые для обнаружения сетевых угроз. Анализ аналогов позволяет определить, какие подходы уже доказали эффективность, какие функции являются обязательными, а какие избыточны для учебного и исследовательского прототипа [10]. В рамках предметной области можно выделить несколько классов готовых систем: - классические сетевые IDS/IPS; - платформы мониторинга и анализа сетевой активности; - host-based решения с функциями обнаружения событий безопасности; - комплексные стеки — (дистрибутивы), — объединяющие несколько инструментов. Ниже рассмотрены наиболее распространенные разработки, близкие по назначению к создаваемому прототипу. Snort — одно из наиболее известных open-source решений класса NIDS/NIPS. Основу Snort составляет сигнатурный механизм, позволяющий описывать правила обнаружения в формализованном виде. Система поддерживает анализ пакетов в реальном времени, ведение журналов, классификацию событий и работу в режимах обнаружения/предотвращения. Преимущества Snort: - зрелая экосистема и длительный опыт промышленного применения; - развитый механизм правил и большое количество готовых сигнатур; - высокая информативность событий. Ограничения для текущего проекта: - значительная сложность конфигурирования для учебного прототипа; - необходимость более глубокой настройки инфраструктуры; - ориентированность на эксплуатацию в зрелом контуре ИБ, а не на легковесный дипломный стенд. Suricata — современная высокопроизводительная IDS/IPS с поддержкой многопоточности, глубокой инспекции протоколов и совместимости с сигнатурами Snort. Система активно используется в ргодиспоп-средах и хорошо подходит для сетей с высокой нагрузкой. Преимущества Suricata: - высокая производительность и масштабируемость; - богатая поддержка протоколов и форматов логирования; - удобная интеграция с внешними системами анализа событий.

10 Ограничения для

текущего проекта: - избыточность функционала относительно целей дипломного прототипа; - более сложный порог входа по настройке и сопровождению; - повышенные требования к инфраструктуре и опыту оператора [11]. Zeek (ранее Bro) представляет иной подход: акцент сделан на поведенческом анализе сетевой активности и формировании подробных событий/логов, а не только на классическом сигнатурном сопоставлении. Это мощный инструмент для сетевой аналитики и расследований. Преимущества Zeek: - гибкая модель событий и глубокая наблюдаемость; - сильная сторона в аналитике и Готепя1с-подходе; - удобство построения сложных сценариев обнаружения. Ограничения для текущего проекта: - более высокий уровень сложности проектирования правил; - значительные трудозатраты на адаптацию под ограниченный учебный контур; - Для первичного прототипа может быть слишком — «тяжелым» концептуально. OSSEC и его развитие Wazuh относятся преимущественно к host-based мониторингу: контроль логов, контроль целостности файлов, реагирование на события на уровне хостов. В современных внедрениях Wazuh часто используется как платформа XDR/SIEM-КОМПОНЕНТ с агентами. Преимущества: - развитая работа с журналами и хостовыми событиями; - централизованная аналитика и уведомления; - удобство интеграции в инфраструктуру с несколькими узлами. Ограничения для текущего проекта: - фокус на епароип//лог-аналитике, а не На легковесном сетевом сниффере; - избыточная архитектурная сложность для учебного одиночного прототипа IDS; - большой объем развертывания и администрирования. Security Onion — комплексный дистрибутив для мониторинга безопасности, включающий набор средств (в том числе Suricata/Zeek и др.) в единой экосистеме. Подходит для построения учебных лабораторий и ЗОС-подобных стендов. Преимущества: - готовый интегрированный стек; - широкий аналитический функционал; - удобство для полноценных стендов безопасности. Ограничения для текущего проекта: - решение уровня целой платформы, а не одного прототипа приложения; - высокая сложность для диплома, где требуется показать собственную разработку; - существенные требования к ресурсам и окружению [12].

11 Таблица 1.1 Сравнительный анализ существующих разработок IDS/IDPS

Решение	Класс	Подход к сильным Ограничения Применимость к системам обнаружению стороны теме диплома
Snort	NIDS/NIP	Преимущество Зрелое Требуется Высокая как S HHO решение, тонкий источник
ПОДХОДОВ	сигнатурный, большая настройки, средняя для правила база сложнее для прямого внедрения правил, быстрого в учебный широкая старта прототип известнос Th Suricata NIDS/NIP	
Сигнатурный +	Высокая Избыточен по Высокая как эталон S протокольный производ	
сложности промышленного анализ, ительность для уровня, средняя многопоточнос ь,		
компактного для реализации “с ть современ учебного нуля” ная проекта архитекту ра,		
масштаби руемость Zeek NIDS/Net Событийно- Глубокая Высокий Высокая для (Bro) work		
поведенческий аналитика порог аналитики, средняя Security анализ трафика , освоения, для		
базового Monitorin детализир сложная дипломного г ованные логика прототипа логи,		
сценариев гибкость OSSEC/ HIDS/XD Анализ логов, Сильная Меньший Средняя: полезно Wazuh R		
событий хоста, хостовая фокус На для интеграции, но (агентный контроль аналитика		
легковесном не как основной подход) целостности, сетевом сетевой прототип централиз		
сниффинге ованный монитори HI Security Комплекс Комбинирован Готовая Тяжелая Высокая		
как Onion ная ный (в составе экосистем инфраструкту ориентир платформ Suricata/Zeek и а,		
удобна рно, не архитектуры, а др.) Для подходит как низкая для монитори лаборатор “легкий”		
реализации в нга (SOC- ий SOC standalone- рамках одной стек) прототип программы HunterEy		
Локальны Пороговые и Е Й сигнатурные (разработ прототип эвристики, ываемый IDS для		
правила прототип) ЛВС 12 Как видно из таблицы 1.1, все рассмотренные решения обладают		
высокой функциональностью и зрелой архитектурой, однако в той или иной мере избыточны для		
задач настоящего дипломного проекта. Промышленные системы — Snort, Suricata и Zeek —		
ориентированы на эксплуатацию в нагруженных production-средах и требуют значительных		
затрат на развёртывание и настройку. Платформы OSSEC/Wazuh и Security Onion, в свою очередь,		
представляют собой комплексные стеки, выходящие за рамки задачи реализации компактного		

сетевого прототипа. Таким образом, ни одно из рассмотренных решений не может быть напрямую использовано в качестве основы проектного прототипа, однако каждое из них послужило источником архитектурных подходов и принципов, реализованных в HunterEyeE. Рассмотренные решения подтверждают, что эффективные IDS строятся вокруг нескольких общих принципов: 1. захват и нормализация трафика; 2. набор формализованных правил обнаружения; 3. генерация и хранение событий безопасности; 4. средства визуализации и последующего анализа. Именно эти принципы — положены в — основу — разрабатываемого прототипа HunterEyeE. При этом в дипломной работе — сознательно выбран умеренный по — сложности путь: не развертывание — тяжелой промышленной платформы, а реализация собственной модульной IDS, ориентированной на локальную сеть и учебный сценарий применения. Таким образом, по результатам анализа существующих разработок приняты следующие проектные решения: - использовать комбинированный подход детекции (пороговые - сигнатурные эвристики); - обеспечить настраиваемость правил и временных окон; - реализовать журналирование и хранение событий в локальной БД; - предоставить оператору графический интерфейс мониторинга; - оставить возможность расширения (добавление новых правил, интеграции, более сложной аналитики). Выбранная концепция обеспечивает баланс между практической ценностью и реализуемостью в рамках дипломного проекта. Прототип сохраняет ключевые свойства IDS, но остается достаточно прозрачным, чтобы продемонстрировать полный цикл разработки, тестирования и дальнейшего масштабирования решения.

1.3 Постановка задачи

По результатам анализа предметной области и существующих разработок сформулирована задача дипломного проекта: разработать прототип системы обнаружения вторжений для локальной сети, обеспечивающий контроль сетевой активности, выявление подозрительных событий, их регистрацию и представление результатов пользователю. Постановка задачи выполняется в логике технического задания и определяет цель работы, перечень задач, требования к системе, границы проекта и ожидаемые результаты [13]. Цель проекта — разработать работоспособный программный прототип IDS HunterEyeE, ориентированный на работу в локальной сети под управлением ОС Windows и предназначенный для выявления типовых сетевых атак и аномалий в режиме, близком к реальному времени. Для достижения поставленной цели необходимо решить следующие задачи: 1. определить требования к прототипу IDS с учетом особенностей локальной сети и условий эксплуатации; 2. спроектировать ~~~ модульную — архитектуру системы и — описать взаимодействие основных компонентов; 3. реализовать подсистему захвата и предварительной обработки сетевого трафика; 4. реализовать подсистему детекции на основе сигнатурных и пороговых правил; 5. реализовать подсистему регистрации событий безопасности и хранения данных; 6. реализовать графический интерфейс оператора для мониторинга и управления параметрами системы; 7. выполнить тестирование и подтвердить работоспособность прототипа на демонстрационных сценариях. Объект разработки — процессы сетевого взаимодействия в локальной сети, влияющие на состояние информационной безопасности. Предмет разработки — программные методы обнаружения вторжений на основе анализа параметров сетевого трафика. В рамках проекта формулируются следующие требования к прототипу IDS. Функциональные требования: - захват сетевых пакетов с выбранного интерфейса локальной сети; - извлечение ключевых атрибутов пакета (время, IP-адреса, порты, протокол, TCP-флаги); - выявление подозрительной активности по набору правил (PoS-подобная активность, сканирование портов, аномальная SYN-активность, обращения к подозрительным портам, события по черному списку IP); - формирование событий безопасности с указанием типа, уровня критичности и описания; - запись событий в журнал и в локальную базу данных SQLite; - отображение текущих и архивных событий через графический интерфейс; - настройка параметров обнаружения (временные окна, пороги, включение/отключение правил); - экспорт результатов в рамках реализованной функциональности. Нефункциональные требования: 14 - работоспособность в среде Windows 10/11; - использование доступных Open-source средств разработки; - устойчивость к ошибкам

обработки входных данных; - модульность архитектуры и возможность расширения набора правил; - приемлемая производительность для учебного или лабораторного сегмента сети; - интерпретируемость результатов детекции для пользователя. Требования к пользовательскому взаимодействию: - отображение состояния работы системы; - просмотр списка выявленных инцидентов; - базовая фильтрация и анализ истории событий; - доступ к настройкам правил и основных параметров IDS; - вывод диагностических сообщений при ошибках. Поскольку разрабатывается именно прототип, в работе принимаются следующие ограничения: анализируется только трафик, доступный выбранному сетевому интерфейсу; набор правил ограничивается типовыми сценариями, достаточными для учебной демонстрации; не реализуется полнофункциональная интеграция с промышленными \$EM/3OAK-платформами; автоматическое предотвращение инцидентов допускается в базовом объеме [13]. Критериями достижения результата считаются: корректный запуск системы в целевой среде, стабильный прием и обработка пакетов, срабатывание реализованных правил на тестовых сценариях, фиксация инцидентов в журнале и базе данных, а также доступность результатов через графический интерфейс. Ожидаемым результатом проектной работы является программный прототип HunterEye, включающий подсистемы захвата, детекции, регистрации, хранения и визуализации событий безопасности, а также набор тестов и описание порядка эксплуатации. Таким образом, постановка задачи определяет не только общий вектор выполнения дипломного проекта, но и конкретный, измеримый и проверяемый результат разработки. В рамках работы предполагается создание модульного прототипа IDS для локальной сети, обеспечивающего полный цикл обработки сетевых инцидентов: от захвата и анализа трафика до регистрации событий, их сохранения и предоставления оператору структурированной информации о выявленных угрозах. Такой подход позволяет рассматривать разрабатываемую систему как законченное программное решение учебно-прикладного характера, обладающее как теоретической, так и практической значимостью.

15.2 Выбор технологий и средств разработки

После определения требований к разрабатываемому прототипу IDS следующим этапом является выбор технологий и программных средств, на основе которых будет построена система. Данный этап имеет принципиальное значение, поскольку от выбранного стека зависит не только ВОЗМОЖНОСТЬ реализации требуемого функционала, но и удобство разработки, тестирования, сопровождения и дальнейшего расширения программного продукта [14]. При выборе технологий для дипломного проекта необходимо учитывать как технические, так и практические факторы. С одной стороны, выбранные средства должны обеспечивать доступ к сетевому трафику, обработку пакетов, хранение событий, построение графического интерфейса и выполнение прикладной логики обнаружения вторжений. С другой стороны, стек разработки должен быть достаточно доступным, понятным и реалистичным для реализации в рамках учебного проекта, ограниченного по времени и ресурсам [15]. Разрабатываемый прототип ориентирован на эксплуатацию в локальной сети под управлением ОС Windows. Это накладывает дополнительные требования к выбору среды исполнения и библиотек. В частности, необходимо обеспечить совместимость с механизмами захвата сетевого трафика в Windows, возможность создания настольного приложения с удобным интерфейсом, а также наличие инструментов для быстрого прототипирования и тестирования. В рамках данного раздела обосновывается выбор языка программирования, среды исполнения, библиотек для работы с сетевым трафиком, средств реализации графического интерфейса, инструментов хранения данных и тестирования. Целью является формирование технологической базы, позволяющей реализовать прототип IDS как законченное — модульное приложение. С практической точки зрения выбор технологий для проекта должен отвечать следующим критериям: — соответствие целям разработки и функциональным требованиям; — доступность библиотек и документации; — возможность быстрого создания и модификации прототипа; — поддержка модульной архитектуры; — совместимость с ОС Windows и локальной средой эксплуатации; — возможность дальнейшего масштабирования и доработки [32]. Таким образом, раздел посвящен не простому перечислению

используемых инструментов, а именно обоснованию их выбора с точки зрения задач дипломного проекта. Это позволяет показать, что архитектурные и программные решения приняты осознанно и согласованы с поставленной задачей разработки прототипа IDS для локальной сети.

2.1 Обоснование выбора языка программирования и среды исполнения

После формулирования требований к прототипу IDS необходимо определить программную основу, на которой будет реализован полный контур работы системы: захват сетевого трафика, извлечение признаков пакетов, применение правил детекции, формирование событий, хранение данных и отображение результатов пользователю. От выбора языка программирования и среды исполнения зависят скорость разработки, удобство сопровождения и возможность проверки корректности работы отдельных модулей. Для рассматриваемого прототипа HunterEye целесообразно выбрать язык, который:

1. позволяет быстро реализовать конвейер обработки пакетов и событий;
2. имеет библиотеки для работы с сетью и пакетами в условиях Windows;
3. поддерживает создание настольного GUI с выводом списков/таблиц и динамическим обновлением данных;
4. обеспечивает удобную модульность (разделение на подсистемы захвата, детекции, хранения и интерфейса);
5. позволяет применять автоматизированное тестирование.

В качестве основного языка программирования выбран Python. Такой выбор обоснован тем, что Python является практичным для дипломных и учебно-прикладных проектов: он сокращает время на реализацию инфраструктуры приложения и позволяет сосредоточиться на логике детекции и проектировании архитектуры. В рамках разработки IDS это важно, так как основная сложность заключается не только в том, чтобы «получать пакеты», но и в том, чтобы корректно нормализовать данные, устойчиво — обрабатывать — ошибки, организовать хранение событий и сделать результаты понятными для оператора [15]. Средой исполнения выступает стандартная среда выполнения Python (интерпретатор версии 3.X) с использованием виртуального окружения для воспроизводимости. Выбор среды исполнения обусловлен тем, что:

- изменения в логике детектора и обработке пакетов можно вносить итеративно и быстро проверять;
- модульная структура упрощает тестирование отдельных частей (например, правил и репозитория, отвечающего за хранение);
- поддержка сторонних библиотек дает возможность реализовать требуемые подсистемы без разработки низкоуровневых компонентов «с нуля» [16].

Поскольку система ориентирована на локальную сеть под Windows, при выборе средств исполнения также учитывалась реализуемость захвата сетевого трафика. В проекте применяется связка, обеспечивающая доступ к пакетам с интерфейса: библиотека для работы с пакетами выбирается таким образом, чтобы она была совместима с драйвером захвата в Windows. Это снижает зависимость от сложной сетевой инфраструктуры и позволяет сохранить фокус именно на логике IDS и прототипировании [17].

2.1 К где по основным компонентам прототипа IDS сопоставлены рассмотренные альтернативы, выбранные средства и кратко обоснована их целесообразность

В рамках дипломного проекта. Таблица 2.1 Сводное обоснование выбора технологий разработки прототипа IDS

Компонент	Рассмотренные альтернативы	Выбранное решение	Обоснование выбора
Язык	C/C++, Java, C#, Go	Python	Быстрая разработка, понятная программистам, модульность, удобная обработка данных и простое сопровождение исполнения проекта
Захват	Raw sockets, Scapy (на Linux), Npcap (на Windows)	Scapy (на Linux), Npcap (на Windows)	Практичная реализуемость в анализ DPI/SDK с высоким порогом входа, Npcap — удобный инструмент для захвата трафика
Пользовательский интерфейс	Tkinter, PyQt/PySide, PySide6	PySide6	Подходит для настольных веб-интерфейсов для Python
Хранение данных	Файлы (CSV/JISON), SQLite, PostgreSQL/MySQL	SQLite	Автономная локальная база без необходимости серверной инфраструктуры, быстрые операции
Тестирование	ручные проверки, unittest, pytest	pytest	Удобно проводить модульное и интеграционное тестирование логики детектора и компонентов хранения

Таблица 2.1 показывает, что для

прототипа IDS HunterEyE сформирован согласованный набор технологий, охватывающий основные этапы обработки данных: от получения и разбора сетевого трафика до отображения событий оператору, их регистрации и проверки корректности работы модулей. Выбранные средства в совокупности закрывают функциональные требования к прототипу при ограниченной инфраструктуре и приемлемой трудоёмкости реализации в рамках дипломного проекта.

Разработка программного кода выполнялась в среде Visual Studio Code (Cursor), что обеспечило удобную работу с модульной структурой проекта, отладкой и запуском тестов. Подводя итог подраздела 2.1, выбор Python и среды исполнения Python 3.x обоснован задачами прототипирования: ускоряется разработка, упрощается модульная организация кода и тестирование отдельных компонентов IDS. Использование виртуального окружения повышает воспроизводимость проекта, а учёт особенностей Windows задаёт логичный переход к обоснованию средств захвата и анализа трафика в последующих подразделах раздела 2.

2.2 Обоснование выбора средств захвата и анализа сетевого трафика

Для прототипа IDS ключевым технологическим звеном является подсистема получения сетевых данных, поскольку качество последующей детекции напрямую зависит от полноты и корректности извлекаемых признаков пакетов. В локальной сети под управлением ОС Windows захват трафика обычно реализуется через специализированный драйвер доступа к пакетам и прикладную библиотеку, которая позволяет декодировать сырые кадры и пакеты в структурированное представление [18]. В качестве базового механизма доступа к пакетам в среде Windows выбран стек, совместимый с Mpcap (современная реализация интерфейса, близкого по назначению к WinPcap/libpcap). Такой подход обусловлен тем, что прикладной код не может устойчиво получать «сырые» пакеты на сетевом интерфейсе без соответствующего уровня поддержки со стороны ОС и драйвера. Для учебного прототипа также важны доступность установки и наличие документации, что снижает порог развёртывания стенда. Для программной обработки пакетов выбрана библиотека Scapy. Она позволяет разбирать сетевой трафик на уровнях 1.2/1.3/1.4, работать с типовыми протоколами (в том числе IPy4, TCP, UDP, ARP) и извлекать поля, необходимые для IDS: адреса источника и назначения, порты, протокол, флаги TCP и другие атрибуты. По сравнению с ручным низкоуровневым парсингом это существенно сокращает трудозатраты и уменьшает риск ошибок при разборе заголовков [14]. В проекте HunterEyE захват организован через AsyncSniffer. Это связано с тем, что требуется запускать sniffing в отдельном потоке и корректно останавливать захват без длительного блокирования, характерного для некоторых простых блокирующих режимов. Такой вариант лучше сочетается с настольным приложением: захват работает параллельно графическому интерфейсу, а обработка пакетов выполняется через функцию обратного вызова.

После получения пакета выполняется нормализация данных: из «сырого» представления формируется внутренняя структура с фиксированным набором полей (время, адреса, порты, протокол, признаки TCP и др.), удобная для применения правил детекции. Такой слой нормализации важен, потому что логика [DS в прототипе опирается не на произвольный пакет, а на единообразное описание сетевого события. Следует учитывать ограничения хостового мониторинга в локальной сети: полнота наблюдения зависит от топологии и от того, какой трафик реально проходит через выбранный интерфейс. Прототип анализирует доступный интерфейсу объём данных и не претендует на охват всей сети как у распределённых промышленных решений. Это соответствует границам дипломной постановки задачи. Таким образом, для Windows обоснована связка Npcap + Scapy с асинхронным режимом захвата: она обеспечивает реализуемость прототипа, достаточную детализацию признаков для правил IDS и приемлемую интеграцию с архитектурой настольного приложения.

2.3 Выбор

В подразделе 2.2 обоснован выбор средств захвата и анализа трафика: для Windows используется драйвер пакетного доступа (Npcap), а для разбора пакетов — Scapy, что упрощает извлечение полей, нужных для правил IDS. Захват выполняется асинхронно, чтобы не блокировать работу приложения. Нужно помнить, что виден только трафик, доступный выбранному интерфейсу. Далее рассматриваются средства интерфейса, хранения данных и тестирования.

средств интерфейса, хранения и тестирования Помимо захвата и анализа сетевого трафика прототип 10% должен обеспечивать удобное представление результатов оператору, надёжное сохранение событий для последующего просмотра и отчётности, а также возможность проверки корректности отдельных модулей. Поэтому в рамках одного подраздела целесообразно совместно обосновать выбор средств для графического интерфейса, хранения данных и автоматизированного тестирования: эти компоненты образуют единый контур «отображение – персистентность – верификация», необходимый для завершённого программного продукта.

2.3.1 Пользовательский интерфейс

Для реализации настольного приложения выбран Py514e6 (официальные Qt-биндинги для Python). Обоснование связано с тем, что прототипу требуется типовой для мониторинга интерфейс: отображение списков/таблиц событий, переход между разделами (например, мониторинг, правила, события, настройки), работа с настройками и визуальная индикация состояния системы. Qt предоставляет зрелые виджеты и механизмы оформления (в том числе таблицы стилей, что снижает трудозатраты по сравнению с «минималистичными» CIL-фреймворками и ускоряет получение аккуратного пользовательского интерфейса [19]. Среди альтернатив (Tkinter, веб-интерфейс) выбор PySide6 обусловлен ориентацией на полноценное десктоп-приложение без необходимости поднимать веб-сервер и без ограничений по интерактивности табличных данных. Для дипломного проекта это согласуется с задачей быстрого прототипирования GUI и дальнейшего сопровождения кода.

2.3.2 Хранение данных и журналирование

Для хранения событий IDS выбрана встраиваемая СУБД SQLite. Причины выбора: автономность (отдельный сервер БД не требуется), простота развёртывания на одной рабочей станции, достаточная скорость для учебного/лабораторного объёма данных, удобство выполнения выборки по времени, IP и уровню события. В прототипе предполагается фиксация инцидентов в табличном виде с индексацией по ключевым полям, что соответствует задачам последующего просмотра истории и формирования отчётности. 20 Альтернативы в виде только текстовых журналов или исключительно файлов CSV/JSON остаются возможными для простейших сценариев, но ухудшают удобство структурированных запросов и фильтрации. Серверные СУБД (PostgreSQL/MySQL) для локального прототипа избыточны и усложняют сопровождение без существенного выигрыша для демонстрационного объёма данных. Дополнительно в системе может использоваться текстовое журналирование как вспомогательный канал фиксации работы компонентов; однако основным носителем структурированных событий для GUI и анализа выступает SQLite.

2.3.3 Тестирование

Для автоматизированной проверки логики выбран фреймворк pytest. Он удобен для написания модульных и интеграционных тестов, поддерживает параметризацию и хорошо подходит для проверки компонентов, критичных для IDS: правил детектора, операций записи/чтения репозитория событий, корректности обработки пограничных случаев. По сравнению с ручным тестированием это повышает воспроизводимость результатов и снижает риск регрессий при доработке правил [20]. Стандартный модуль unittest также применим, однако для типичного Python-проекта pytest часто снижает шаблонность тестов и упрощает поддержку набора проверок по мере роста проекта. Таким образом, связка PySide6 + SQLite + pytest соответствует задачам прототипа HunterEye: обеспечить наглядный интерфейс мониторинга, локальное структурированное хранение событий и устойчивую верификацию ключевой логики без избыточной инфраструктуры. Выбор средств построения интерфейса, хранения данных и тестирования позволяет завершить формирование технологического стека прототипа. Оператор получает удобное настольное приложение с наглядным отображением сетевых событий, все зафиксированные инциденты сохраняются в локальной базе данных и доступны для последующего анализа и фильтрации, а ключевые модули системы могут проверяться с помощью автоматических тестов, что обеспечивает воспроизводимость результатов и упрощает сопровождение кода. Совокупность выбранных технологий образует целостный и сбалансированный стек, в котором каждый компонент решает конкретную задачу и не вступает в противоречие с остальными. Такой подход минимизирует архитектурные риски на этапе

реализации и позволяет сосредоточить усилия на логике обнаружения, а не на интеграции разнородных инструментов. Сформированная технологическая основа создаёт необходимые предпосылки для перехода к проектированию системы и детальному описанию её реализации в разделе 3. 21 3 Проектирование и реализация прототипа В данном разделе рассматривается практическая часть дипломного проекта — переход от сформулированных требований и выбранного технологического стека к работающему программному решению. Если в предыдущих главах были определены границы задачи, проведён анализ аналогов и обоснован выбор инструментов, то здесь основной акцент сделан на том, как эти решения реализованы в прототипе HunterEye на уровне архитектуры, программных модулей и пользовательского взаимодействия. Цель главы — показать, что разработка IDS выполнена как целостная инженерная система, где каждый элемент решает конкретную задачу и встроен в общий процесс обработки сетевых событий. Прототип проектируется не как набор разрозненных скриптов, а как модульное приложение с предсказуемым поведением и возможностью дальнейшего расширения. В рамках главы описывается полный функциональный контур работы системы: — получение сетевого трафика с выбранного интерфейса; — преобразование входных данных в единый внутренний формат; — применение правил обнаружения и формирование инцидентов; — регистрация результатов в журнале и базе данных; — визуализация состояния IDS и обнаруженных событий в интерфейсе оператора; — конфигурирование параметров работы без изменения исходного кода; — запуск и эксплуатация прототипа в целевой среде. Практическая реализация выполнена с учётом ограничений дипломного проекта: автономная работа на локальной машине, отсутствие обязательной серверной инфраструктуры, воспроизводимость развёртывания в Windows и наглядность демонстрации на защите. Такой подход позволяет достичь баланса между технической полнотой и реализуемостью в учебных условиях. Ключевым принципом при разработке выбран модульный подход. Это означает, что подсистемы захвата, анализа, хранения и визуализации логически отделены друг от друга и взаимодействуют через понятные точки обмена данными. Модульность снижает связанность кода, облегчает локальные изменения и повышает тестируемость. Например, корректировка порогов детекции или расширение набора правил не требует переработки графического интерфейса, а изменения в визуальном представлении не затрагивают механизм захвата пакетов [21]. С инженерной точки зрения проектирование и реализация прототипа опираются на следующие требования качества: 1. Непрерывность обработки. Система должна стабильно принимать и анализировать поток сетевых данных в режиме, близком к реальному времени. 2. Устойчивость. Ошибки отдельных этапов (например, временные проблемы захвата) не должны приводить к отказу всего приложения. 22 3. Наблюдаемость. Пользователь должен видеть состояние работы IDS, состав событий и результаты детекции в понятной форме. 4. Конфигурируемость. Ключевые параметры должны изменяться без изменения программной логики ядра. 5. Расширяемость. Архитектура должна позволять добавлять правила, сценарии реагирования и дополнительные каналы анализа [22]. Отдельное значение в главе придаётся связке «проектирование – реализация». Проектные решения рассматриваются не как абстрактные схемы, а как основания для конкретного программного кода. Такой способ изложения важен для дипломной работы, поскольку подтверждает, что заявленные архитектурные принципы действительно реализованы в рабочем прототипе и отражены в структуре проекта. Важной частью главы является также описание эксплуатационного контура. Для IDS недостаточно только корректной детекции; система должна быть удобной в практическом применении: запускаться в предсказуемом режиме, поддерживать базовую диагностику, хранить историю инцидентов и обеспечивать оператору минимально необходимый уровень управления. Поэтому в рамках реализации рассматриваются не только алгоритмы обнаружения, но и вопросы конфигурации, журналирования и представления результатов [23]. Итогом главы должно стать обоснованное подтверждение того, что разработанный прототип HunterEye реализует полный цикл IDS в пределах поставленной задачи: от получения пакетов до

отображения инцидентов и их сохранения для последующего анализа. 3.1 Архитектура системы и состав модулей Архитектура прототипа HunterEyeE спроектирована как модульная, с разделением на функциональные подсистемы, каждая из которых отвечает за отдельный этап обработки данных. Такой подход выбран осознанно: для IDS важно не только обнаруживать события, но и обеспечивать устойчивый и понятный цикл работы — от захвата трафика до отображения инцидентов оператору. Модульная структура позволяет локально изменять и развивать отдельные части приложения без полной переработки всей системы. В основе архитектуры лежит конвейерная модель. Сетевой трафик поступает в подсистему захвата, далее проходит нормализацию, затем передается в модуль детекции, который принимает решение о наличии признаков атаки или аномалии. Результаты анализа регистрируются в журнале и базе данных, после чего становятся доступными для просмотра через графический интерфейс. При необходимости события передаются в модуль реагирования [24]. Для реализации такой логики система должна удовлетворять нескольким условиям: — захват трафика не должен блокировать интерфейс пользователя; 23 — формат данных между модулями должен быть единообразным; — правила детекции должны быть настраиваемыми; — хранение событий должно поддерживать фильтрацию и последующий анализ; — интерфейс должен отображать состояние системы в наглядной форме. С архитектурной точки зрения HunterEyeE можно представить как совокупность пяти уровней: 1. Уровень захвата данных — получение пакетов с сетевого интерфейса. 2. Уровень нормализации — приведение данных к внутренней структуре события. 3. Уровень аналитики (детекции) — применение правил и формирование инцидентов. 4. Уровень хранения и журналирования — фиксация событий и служебной информации. 5. Уровень представления и управления — GUI, настройки, контроль работы IDS [32]. Наличие этих уровней обеспечивает предсказуемость потока данных и упрощает анализ поведения системы на этапе тестирования.

Scapy AsyncSniffer правила + эвристики Модуль захвата Модуль детекции Модуль реагирования Пользовательский (Capture) (Detection Engine) (Response) интерфейс (GUI) packet_sniffer.py detector.py response_service.py main_window.py firewall_adapter.py PySide6 | У Модуль хранения (Storage) database.py (SQLite) logger.py repository.py Рисунок 3.1 — Структурная схема архитектуры прототипа IDS HunterEyeE Как видно из рисунка 3.1, представленная схема отражает конвейерную логику обработки данных: каждый уровень архитектуры имеет чётко определённую ответственность и взаимодействует с соседними уровнями через стандартизированные интерфейсы передачи данных.

3.1.1 Состав программных модулей Практическая реализация архитектуры отражена в структуре исходного кода проекта. Ключевые модули выполняют следующие функции. 24 1. Точка входа приложения (main.py) Файл запускает приложение и подготавливает окружение. На этом уровне выполняется инициализация пути к внутренним пакетам, запуск основного GUI- модуля, а также подключение вспомогательных компонентов. Централизация запуска в одной точке упрощает эксплуатацию и снижает риск ошибок конфигурации. 2. Подсистема захвата трафика (packet sniffer.py) Модуль построен на асинхронном механизме приема пакетов. Асинхронный режим критически важен для десктопного приложения, потому что позволяет выполнять захват параллельно с работой интерфейса. Каждый полученный пакет передается в обработчик, где выделяются признаки для последующей детекции. 3. Подсистема детекции (detector.py) Это аналитическое ядро IDS. Здесь реализованы правила и эвристики, связанные с анализом интенсивности трафика, сканированием портов, SYN- активностью и другими признаками. Подсистема опирается на временные окна и пороговые значения, что делает детекцию воспроизводимой и понятной. Отдельно предусмотрена статистика срабатываний для отображения в интерфейсе и для отчётного анализа. 4. Подсистема конфигурации правил (rules service) Параметры правил вынесены в конфигурационные файлы. Это позволяет изменять поведение IDS без редактирования кода детектора. В учебном и демонстрационном сценарии это особенно полезно: можно оперативно менять пороги и проверять влияние параметров на чувствительность системы. 5. Подсистема хранения (database.py, repository.py) Хранение

событий реализовано на SQLite. База данных используется для накопления инцидентов, фильтрации по ключевым полям и последующего анализа. Такой подход обеспечивает автономность решения и избавляет от необходимости разворачивать отдельный сервер БД. 6. Подсистема журналирования (logger.py) Журнал фиксирует ход работы системы и служебные события. Это важно для диагностики нестандартных ситуаций и подтверждения корректности работы при тестировании. 7. Подсистема реагирования (response service, firewall adapter) В проекте предусмотрен базовый слой реакций на инциденты. Он позволяет в рамках прототипа расширить систему от классической IDS к сценариям с элементами предотвращения. 8. Подсистема пользовательского интерфейса (main window.py) GUI обеспечивает оператору доступ к событиям, настройкам и состоянию системы. Визуальный слой отделен от логики захвата и детекции, что позволяет дорабатывать интерфейс без вмешательства в аналитическое ядро [25].

3.1.2 Поток данных внутри системы Системный поток обработки можно описать следующей последовательностью: 1. Сниффер получает сетевой пакет с выбранного интерфейса. 2. Пакет преобразуется к внутренней структуре признаков (PacketInfo- подобная модель). 3. Модуль детекции применяет набор активных правил. 4. При срабатывании формируется событие инцидента с типом, уровнем и описанием. 5. Событие записывается в журнал и в БД. 6. GUI обновляет отображение списка событий и телеметрии. Данный поток важен для архитектурной целостности: каждый этап имеет четкую ответственность а обмен между — этапами строится на стандартизированных данных. Это снижает вероятность логических ошибок, особенно при расширении системы. Режим | Обнаружение @ Остановлено настройки Тип содержит Уровень Все Период с по Применить фильтр. Сбросить уровень описание 250 HIGH Low o LOW Low Загрузить ещё 200 | Показано: 7 (лимит 300 Рисунок 3.2 — Отображение потока зарегистрированных событий в интерфейсе HunterEyeE Как видно из рисунка 3.2, интерфейс отображает зарегистрированные события в табличном виде с указанием типа инцидента, уровня угрозы и временной метки, что обеспечивает оператору возможность оперативного анализа происходящего в сети. 3.1.3 Архитектурные решения, принятые в проекте При проектировании были приняты решения, влияющие на устойчивость и сопровождаемость прототипа: 1. разделение ответственности модулей: каждый компонент выполняет ограниченный набор функций, что упрощает локализацию ошибок и внедрение изменений; 2. асинхронная обработка захвата: выбор асинхронного механизма устраняет зависимость COT от скорости приёма пакетов и повышает пользовательскую отзывчивость; 3. единая внутренняя модель события: нормализация пакета до фиксированной структуры упрощает применение правил и тестирование; 4. конфигурируемость правил: параметры не зашиты жёстко в код, что позволяет калибровать систему под разные условия ЛВС; 26 5. автономное хранение: SQLite обеспечивает достаточную функциональность без внешней инфраструктуры, что соответствует целям прототипа; 6. готовность к расширению: архитектура допускает добавление новых правил, каналов экспорта и вариантов реакций без смены базовой структуры. 3.1.4 Ограничения архитектуры Как и любой учебно-практический прототип, система имеет ограничения, которые важно зафиксировать уже на уровне архитектуры: — наблюдается только трафик, доступный выбранному интерфейсу; — глубина анализа протоколов ограничена реализованными правилами; — реакция на инциденты представлена в базовом объеме; - система не является распределенной и не заменяет ЗОС-платформы [26]. Эти ограничения не противоречат целям дипломного проекта, а отражают его границы: создать работоспособный и расширяемый прототип IDS для локальной сети, а не промышленный комплекс мониторинга. 3.1.5 Роль интерфейса в архитектуре Интерфейс в HunterEyeE выполняет не декоративную, а эксплуатационную функцию. Через GUI оператор: — видит текущий статус работы IDS; — отслеживает срабатывания правил; — просматривает историю событий; — изменяет параметры конфигурации; — контролирует режим функционирования системы. Таким образом, COT является частью архитектуры управления системой, а не отдельным «дополнением». 1 968 кадров с интерфейса: 1 979 (ERY EY Рисунок 3.3 — Главное окно прототипа HunterEyeE в режиме

мониторинга 27 Как видно из рисунка 3.3, главное окно прототипа предоставляет оператору доступ ко всем ключевым функциям системы в едином пространстве: мониторинг текущего состояния IDS, просмотр событий безопасности и управление параметрами работы выполняются без переключения между отдельными утилитами. В результате проектирования сформирована модульная архитектура прототипа HunterEyeE, обеспечивающая полный цикл IDS: получение сетевых данных, аналитическую обработку, фиксацию инцидентов и их визуализацию. Принятая структура обеспечивает баланс между функциональной полнотой и реализуемостью в рамках дипломного проекта, а также создает техническую основу для последующей детализации реализации подсистем в подразделе 3.2.

3.2 Реализация подсистем захвата, детекции, журналирования и хранения

В данном подразделе рассматривается реализация ключевых подсистем прототипа HunterEyeE, обеспечивающих основной рабочий цикл IDS: получение сетевых данных, выявление признаков угроз, фиксацию событий и сохранение результатов для последующего анализа. В отличие от архитектурного описания, приведенного в 3.1, здесь акцент сделан на прикладной реализации логики и взаимодействии модулей в процессе эксплуатации. Основу работы системы составляет последовательная обработка трафика. Сначала выполняется захват пакетов с выбранного интерфейса, затем извлекаются необходимые признаки, после чего данные передаются в модуль детекции. При срабатывании правил формируется событие безопасности, которое одновременно фиксируется в журнале, записывается в базу данных и становится доступным в пользовательском интерфейсе.

3.2.1 Реализация подсистемы захвата

Подсистема захвата реализована в асинхронном режиме, что позволяет получать пакеты без блокировки работы графического интерфейса. Такое решение особенно важно для настольного приложения: пользователь должен иметь возможность управлять режимом работы IDS и просматривать события независимо от текущей интенсивности трафика [27]. В процессе захвата из пакета извлекаются основные признаки, используемые модулем детекции: — IP-адрес источника и назначения; — используемый протокол; — порты (для TCP/UDP); — флаги TCP; — отметка времени получения пакета. Полученные данные приводятся к унифицированному внутреннему формату, что позволяет применять единые правила обработки независимо от исходного 28 типа пакета. Наличие отдельного слоя ~ нормализации — упрощает масштабирование и уменьшает вероятность логических ошибок в правилах.

3.2.2 Реализация подсистемы детекции

Модуль детекции реализует комбинированный подход к обнаружению: пороговые правила по временным окнам и сигнатурно-эвристические признаки подозрительной активности. В рамках прототипа обрабатываются типовые сценарии локальной сети, включая: — аномальный рост интенсивности пакетов; — признаки сканирования портов; — повышенную SY №-активность; — обращения к подозрительным портам; — события по дополнительным правилам, включенным в конфигурации [28]. Ключевой особенностью реализации является конфигурируемость правил. Параметры обнаружения вынесены из ядра в отдельный слой настроек, что позволяет менять чувствительность системы без редактирования исходного кода. Это важно как для эксплуатационного применения, так и для тестирования: оператор может калибровать пороги и окна анализа под конкретный характер трафика локальной сети. Настройка параметров детекции выполняется через отдельный интерфейс управления правилами.

HIGH MEDIUM HIGH HIGH

Добавить правило Удалить правило Применить и перезагрузить

Рисунок 3.4 — Окно управления правилами обнаружения

Представленный на рисунке 3.4 интерфейс подтверждает, что в системе реализована управляемая модель детекции: включение и отключение правил, а также изменение пороговых параметров выполняется в штатном режиме без вмешательства в программную реализацию ядра.

3.2.3 Реализация журналирования и хранения 29

Подсистема журналирования фиксирует как инциденты безопасности, так и служебные события приложения. Это обеспечивает наблюдаемость системы и позволяет анализировать не только факты срабатывания правил, но и состояние работы IDS в процессе эксплуатации. Для долговременного хранения используется SQLite. Выбор данной СУБД обусловлен

автономностью, простотой развёртывания и достаточной производительностью для задач прототипа. Структурированное хранение **событий позволяет: — выполнять фильтрацию по времени, уровню и IP; — формировать выборки для интерфейса; — поддерживать экспорт и отчётные операции; — сохранять историю между запусками приложения.** Связка «журнал + БД» реализует два уровня фиксации: оперативный (для диагностики) и аналитический (для последующего разбора инцидентов). Такой подход повышает практическую ценность прототипа и делает результаты его работы воспроизводимыми.

3.2.4 Интеграция подсистем в единый цикл обработки

Функциональная целостность НишетЕуЕ обеспечивается тем, что подсистемы захвата, детекции и хранения работают как единый конвейер:

1. Пакет поступает из модуля захвата.
2. Выполняется извлечение и нормализация признаков.
3. Модуль детекции применяет активные правила.
4. При срабатывании формируется событие безопасности.
5. Событие фиксируется в журнале и записывается в SQLite.
6. Результат становится доступным в интерфейсе оператора.

Именно наличие этого непрерывного цикла подтверждает, что прототип реализует **практический функционал IDS, а не отдельные изолированные компоненты.** Таким образом, в подразделе 3.2 показано, что в HunterEуЕ реализованы основные подсистемы ядра: асинхронный захват трафика, настраиваемая детекция, журналирование и структурированное хранение событий. Принятые решения обеспечивают устойчивую работу прототипа в локальной сети и создают основу для дальнейшего описания пользовательской части, конфигурации и развёртывания в подразделе 3.3.

3.3 Интерфейс, конфигурация и развёртывание

В данном подразделе рассматриваются элементы реализации, обеспечивающие практическую — эксплуатацию прототипа HunterEуЕ: пользовательский интерфейс, механизм конфигурирования И порядок развёртывания системы. В отличие от подраздела 3.2, где основной акцент сделан на обработке трафика и детекции, здесь внимание сосредоточено на том, **30** как оператор управляет IDS и как приложение подготавливается к запуску в рабочей среде. Пользовательский интерфейс реализован как настольное приложение на базе PySide6. Такой формат выбран для обеспечения удобного локального мониторинга без дополнительной серверной инфраструктуры. Интерфейс предоставляет оператору доступ к базовым сценариям эксплуатации: запуск и остановка мониторинга, просмотр событий, настройка правил, управление параметрами работы и выполнение сервисных операций (фильтрация, экспорт, работа с блокировками) [28]. Структурно интерфейс организован по вкладкам, что позволяет разделить функции по назначению и повысить удобство работы. Такой подход особенно важен для IDS, где в одном приложении совмещаются задачи оперативного контроля, анализа инцидентов и конфигурирования системы. Разделение на функциональные зоны снижает перегруженность экрана и упрощает выполнение типовых операций. Ключевую роль в эксплуатационном контуре играет конфигурация. В прототипе реализован механизм изменения параметров без правки исходного кода. Пользователь может задавать рабочие настройки, влияющие на поведение системы в конкретной локальной сети: выбирать режим функционирования, управлять параметрами обновления интерфейса и согласовывать параметры захвата с доступным сетевым интерфейсом. Это делает IDS адаптируемой к различным условиям использования и позволяет калибровать систему под конкретный тестовый стенд [29]. Для подтверждения реализации конфигурационного контура в работе приводится окно настроек приложения.

У | Обновить

Рисунок 3.5 — Окно настройки параметров работы прототипа HunterEуЕ

Представленный на рисунке 3.5 экран иллюстрирует, что управление параметрами реализовано в штатном режиме и интегрировано в общий **31** интерфейс системы. Наличие отдельного раздела настроек повышает сопровождаемость прототипа и снижает вероятность ошибок при эксплуатации, поскольку ключевые параметры доступны оператору централизованно. Развёртывание HunterEуЕ выполняется в среде Windows и включает установку программных зависимостей, подготовку драйвера захвата трафика, задание рабочих параметров и контроль корректной регистрации событий в интерфейсе. Такой порядок не требует сложной инфраструктурной подготовки и может быть воспроизведён на стандартной рабочей станции, что соответствует

учебно-прикладному характеру проекта. Практическим преимуществом выбранного подхода является автономность решения: система функционирует без подключения внешних серверных компонентов, что снижает порог внедрения и упрощает демонстрацию прототипа в условиях лабораторного стенда. Оператор получает единый интерфейс управления, объединяющий базовые параметры мониторинга и инструменты контроля состояния приложения. С эксплуатационной точки зрения реализованная модель интерфейса и конфигурирования обеспечивает: — управляемость процесса мониторинга на всех этапах работы IDS; — прозрачность текущего состояния системы и доступность оперативной телеметрии; — возможность изменения параметров без редактирования исходного кода; — воспроизводимость запуска и настройки на целевой платформе. Централизованная настройка — параметров повышает — устойчивость эксплуатации: оператор может адаптировать работу прототипа под особенности конкретной локальной сети, не нарушая целостность программной реализации. Это особенно значимо для IDS, поскольку точность срабатываний напрямую зависит от корректной калибровки порогов и режимов обработки трафика. Для повышения удобства эксплуатации в интерфейсе реализован журнал действий оператора и системных сообщений. Фиксация изменений режима работы, операций экспорта и сообщений об ошибках позволяет быстро восстановить последовательность действий при анализе инцидентов и отладке системы, повышая её наблюдаемость и снижая время на диагностику нештатных ситуаций. Отдельное практическое значение имеет возможность работы в режимах обнаружения и предотвращения. Переключение режимов позволяет продемонстрировать различие между пассивным мониторингом и активной реакцией на инциденты, расширяя прикладную ценность HunterEyeE как основы для дальнейшего развития в сторону полноценных сценариев IDPS [30]. Таким образом, в подразделе 3.3 подтверждена практическая готовность прототипа к использованию в локальной среде: интерфейс поддерживает основные рабочие сценарии оператора, конфигурационный контур обеспечивает адаптацию параметров под условия эксплуатации, а процедура развёртывания остаётся простой и воспроизводимой.

32 4 Тестирование Данный раздел посвящен проверке работоспособности прототипа IDS HunterEyeE в условиях, близких к его целевому использованию в локальной сети. После этапов анализа, выбора технологий и реализации необходимо подтвердить, что система выполняет заявленные функции: принимает трафик, применяет правила детекции, фиксирует события и корректно отображает результаты оператору. Цель тестирования — определить, насколько разработанный прототип соответствует поставленным требованиям и может использоваться как учебно-прикладной инструмент мониторинга. В рамках проверки оцениваются как отдельные функции системы, так и целостная работа конвейера обработки событий [30]. Тестирование выполняется последовательно, по следующим направлениям: — корректность запуска и инициализации основных модулей; — стабильность работы захвата сетевого трафика; — корректность срабатывания правил детекции; — запись инцидентов в журнал и базу данных; — корректность отображения событий и телеметрии в интерфейсе; — устойчивость системы При типовых — действиях — оператора (запуск/остановка, фильтрация, экспорт, изменение параметров). Практическая значимость данной главы заключается в ТОМ, что она подтверждает реальную работоспособность HunterEyeE как целостного программного решения, а не только как набора реализованных модулей. Результаты тестирования позволяют оценить текущий уровень готовности прототипа, определить его эксплуатационные границы и сформулировать направления дальнейшего развития. В рамках испытаний особое внимание уделяется воспроизводимости. Для каждого тестового сценария фиксируется ожидаемый результат и сопоставляется с фактическим поведением системы. Такой подход позволяет получить объективную оценку качества реализации и исключить выводы, основанные только на единичной демонстрации. Отдельно следует отметить, что тестирование IDS в учебном проекте имеет специфику: часть проверок проводится на моделируемых сценариях сетевой активности, а часть — в условиях реального фоновых трафика локального сегмента. Это позволяет одновременно оценить как

контролируемые срабатывания правил, так и поведение системы в «живой» среде [31]. По итогам главы должны быть получены ответы на три ключевых вопроса: 1. Работает ли прототип стабильно в целевой среде? 2. Корректно ли реализованы базовые функции обнаружения и регистрации событий? 3. Какие ограничения остаются и какие направления развития являются приоритетными? 3.3 Таким образом, раздел 4 завершает техническую часть дипломного проекта и формирует основание для итогового заключения: подтверждает, что разработанный прототип IDS HunterEye пригоден для демонстрационного и учебно-практического применения в локальной сети. 4.1 План, среда и виды испытаний Для проверки работоспособности прототипа HunterEye был сформирован план испытаний, охватывающий как отдельные функции системы, так и интегрированный цикл обработки событий. Тестирование проводилось в среде, соответствующей целевому сценарию эксплуатации: локальная сеть, ОС Windows, запуск настольного приложения с активированным захватом трафика [21]. Испытания являются неотъемлемой частью разработки любой системы безопасности, поскольку именно на этом этапе подтверждается соответствие реализованного решения исходным требованиям. Для прототипа класса IDS это особенно важно: система должна не только корректно запускаться, но и стабильно выполнять аналитические функции в условиях реального сетевого трафика.

Отсутствие надлежащего тестирования не позволяет объективно оценить практическую ценность разработки и выявить потенциальные уязвимости в логике обнаружения. Основная задача данного этапа — подтвердить, что разработанный прототип корректно выполняет ключевые функции IDS: — приём и обработку сетевого трафика; — срабатывание правил детекции при наличии характерных признаков; — регистрацию инцидентов в журнале и базе данных; — отображение результатов в интерфейсе; — устойчивую работу при типовых действиях оператора. 4.1.1 Среда испытаний Выбор среды тестирования обусловлен целевым назначением прототипа: система разрабатывалась как настольное приложение для мониторинга локальной сети под управлением ОС Windows. Воспроизведение именно этой конфигурации при испытаниях обеспечивает максимальное соответствие между условиями тестирования и реальной эксплуатацией. Применение иной программной или аппаратной среды могло бы привести к результатам, не отражающим фактическое поведение системы в целевом окружении.

Испытания выполнялись в следующей конфигурации: — операционная система: Windows; — язык и среда выполнения: Python 3.x; — библиотеки и зависимости: в соответствии с requirements.txt; — драйвер захвата трафика: Npcap; — хранилище событий: SQLite; 3.4 — пользовательский интерфейс: Py514e6-приложение HunterEye [31]. Указанная конфигурация обеспечивает воспроизводимость тестов и соответствует условиям, в которых прототип планируется демонстрировать и использовать в учебных целях. Все испытания проводились на одной и той же машине без изменения конфигурации в ходе тестирования, что исключает влияние аппаратных различий на результаты проверок. 4.1.2 План испытаний Планирование испытаний выполнялось исходя из функциональной структуры прототипа: каждый этап проверки соответствует одному или нескольким модулям системы. Такой подход позволяет не только зафиксировать общий результат, но и локализовать возможные отклонения в конкретной подсистеме. Поэтапная структура также упрощает повторное тестирование при внесении изменений в отдельные компоненты. Тестирование проводилось поэтапно: 1. подготовка и запуск: проверка корректного старта приложения, доступности интерфейса и выбора сетевого адаптера; 2. проверка захвата трафика: контроль появления телеметрии (рост счётчиков пакетов, обновление графиков, активность сеанса захвата); 3. проверка детекции: генерация тестовой активности и подтверждение срабатывания реализованных правил (например, SYN flood, подозрительные порты, PoS-подобные сценарии); 4. проверка журналирования и хранения: подтверждение записи инцидентов в таблицу событий и в локальную базу данных; 5. проверка пользовательских функций: фильтрация, экспорт, работа с настройками, повторный запуск и остановка без критических сбоев; 6. фиксация результатов: сопоставление ожидаемого и фактического поведения системы. 4.1.3 Виды испытаний Выбор видов испытаний определялся

архитектурой прототипа и характером проверяемых свойств. Поскольку HunterEyeE представляет собой модульную систему с чётко разграниченными подсистемами, наиболее целесообразным является сочетание проверок на разных уровнях: от изолированного тестирования отдельных компонентов до — комплексной | проверки пользовательских сценариев. Только совокупность этих уровней позволяет получить полноценную картину работоспособности системы. В рамках подраздела применялись три уровня проверки: — модульные проверки — корректность отдельных компонентов (детекция, хранение, конфигурация); — Интеграционные проверки — корректность передачи данных между модулями; - функциональные проверки -- выполнение типовых пользовательских сценариев в интерфейсе [32].

4.1.4 Критерии успешности

35 Для обеспечения объективности оценки результатов критерии определены успешности каждого сценария. заранее были Применения формализованных критериев исключает субъективную трактовку результатов и позволяет однозначно квалифицировать соответствующее или не соответствующее ожиданиям. Сценарий считался успешным, если: — функция выполнялась без критического отказа; — фактический результат соответствовал ожидаемому; — полученные данные были доступны в интерфейсе и/или хранилище; — результат воспроизводился при повторном запуске сценария.

поведение системы как Таблица 4.1 План и критерии испытаний прототипа HunterEyeE №

Проверяемая	Тестовый	Ожидаемый	Критерий	функция сценарий
результат успешности	1	Запуск приложения		Старт main.py, Приложение
открытие GUI	запускается без	доступно, элементы критических ошибок		управления активны
2	Захват трафика	Запуск	Увеличиваются	Есть динамика мониторинга на счётчики пакетов, метрик и графиков
выбранном	обновляется во время сеанса	интерфейсе телеметрия	3	Детекция по Генерация
Формируются	В таблице событий	правилам тестовой сетевой		события с типом и ПОЯВЛЯЮТСЯ активности
уровнем угрозы	корректные срабатывания	4	Журналирование	Выполнение Действия
Записи журнала операций	в фиксируются в отражают	интерфейсе	журнале	последовательность приложения действий
5	Хранение в БД	Срабатывание События	История правил и сохраняются и инцидентов	перезапуск доступны после читается из SQLite приложения
перезапуска	6	Фильтрация	Применение	Список событий Отображаются событий фильтров по сужается по только релевантные уровню ЛР/период условиям фильтра
записи	У	7	Экспорт данных	Экспорт в Создаются файлы с Файлы доступные выгруженными формируются и форматы событиями содержат корректные данные
8	Настройки и	Изменение	Параметры	Система конфигурация параметров, применяются и запускается и повторный запуск сохраняют работает с новыми работоспособность настройками

36 Анализ таблицы 4.1 показывает, что план испытаний охватывает все ключевые функции прототипа HunterEyeE: запуск и управление приложением, захват трафика, детекцию, журналирование, хранение данных и пользовательские операции в интерфейсе. Для каждой проверяемой функции **заданы конкретный сценарий, ожидаемый результат и измеримый критерий успешности, что обеспечивает воспроизводимость проверки и объективность последующей оценки.** Таким образом, в подразделе 4.1 сформированы среда, план и критерии испытаний, позволяющие объективно оценить качество реализации прототипа IDS. На следующем этапе (4.2) выполняется анализ фактических результатов тестирования и фиксируются ограничения текущей версии системы.

4.2 Результаты испытаний и ограничения прототипа

В подразделе 4.1 был сформирован план испытаний и определены критерии успешности для ключевых функций системы. В данном подразделе приводятся фактические результаты проверок прототипа HunterEyeE, анализируется соответствие ожидаемому поведению, а также фиксируются ограничения текущей реализации и направления дальнейшего развития. Испытания выполнялись в условиях локальной сети и охватывали как отдельные функциональные узлы, так и сквозной цикл работы IDS. Особое внимание уделялось стабильности сценария «запуск -> захват -> детекция -> регистрация -> отображение», поскольку именно этот цикл определяет практическую пригодность прототипа.

4.2.1 Общая оценка результатов

По итогам проверок установлено, что прототип корректно выполняет основные задачи, поставленные в дипломной работе: - приложение стабильно запускается и

предоставляет доступ к основным вкладкам интерфейса; - захват трафика выполняется в рабочем режиме, телеметрия обновляется; - правила детекции формируют события при наличии характерной тестовой активности; - события отображаются в таблице интерфейса и сохраняются в локальной базе; - доступны эксплуатационные функции (фильтрация, экспорт, настройки, переключение режима); - повторные запуски не приводят к потере базовой работоспособности. Таким образом, прототип подтвердил функциональную состоятельность как учебно-прикладная [IDS для локального сегмента. 4.2.2 Детализация результатов по функциональным блокам 4.2.2.1 Запуск и базовая инициализация Проверка запуска показала, что GUI открывается корректно, основные элементы управления доступны, переключение вкладок выполняется без сбоев. 37 При старте и остановке мониторинга приложение сохраняет устойчивость, что подтверждает корректную организацию жизненного цикла фоновых процессов. Отдельно отмечено, что пользовательский сценарий запуска интуитивен: оператору доступен выбор режима и интерфейса, после чего можно сразу переходить к мониторингу. 4.2.2.2 Захват трафика и телеметрия На этапе захвата фиксировалось обновление счетчиков пакетов и графиков активности. Это подтверждает, что подсистема приема данных интегрирована с визуальным слоем и корректно передает телеметрию в интерфейс. Асинхронная организация захвата показала практическую эффективность: во время активного приема трафика приложение оставалось управляемым, команды оператора (переход между вкладками, фильтрация, просмотр событий) не блокировались. 4.2.2.3 Детекция и срабатывание правил Проверка детекции проводилась на моделируемых сценариях и фоновом трафике. Были получены срабатывания по реализованным типам правил (в том числе события уровня LOW/MEDIUM/HIGH), что подтверждает работоспособность rule-based подхода в текущей конфигурации. Изменение параметров правил через интерфейс влияло на чувствительность обнаружения ожидаемым образом. Это подтверждает — корректность конфигурационного контура: настройки действительно участвуют в логике принятия решений. 4.2.2.4 Журналирование и хранение События фиксировались в интерфейсной таблице и сохранялись в SQLite. После перезапуска приложения ранее зарегистрированные записи оставались доступными, что подтверждает корректную работу персистентного слоя. Наличие журнала действий также упростило диагностику пользовательских операций (переключение режимов, экспорт, очистка, запуск/остановка), что положительно влияет на эксплуатационную прозрачность системы. 4.2.2.5 Пользовательские функции Фильтрация по параметрам (уровень, IP, интервал времени) выполнялась в рабочем режиме: при задании условий отбора интерфейс корректно отображал только соответствующие записи, исключая нерелевантные события из текущего представления. Это подтверждает, что подсистема хранения обеспечивает не только накопление данных, но и их структурированное извлечение по запросу оператора. Экспорт событий в реализованные форматы также обрабатывал корректно: сформированные файлы содержали полный набор полей инцидента и были пригодны для последующего анализа за пределами приложения. Наличие функции экспорта повышает практическую ценность прототипа, поскольку позволяет передавать результаты мониторинга во внешние инструменты обработки данных или использовать их в отчетных целях. Кроме того, поддержка нескольких форматов выгрузки обеспечивает совместимость с широким 38 спектром аналитических платформ и упрощает интеграцию системы в существующие рабочие процессы организации.

Таблица 4.2 Фактические результаты тестирования функций прототипа HunterEye № |
Проверяемая функция Ожидаемый Фактический Статус результат результат 1 Запуск приложения и COT | Приложение Запуск выполнен, Успешно запускается без вкладки и элементы критических ошибок | управления доступны 2 Запуск/остановка Система корректно Команды Успешно мониторинга стартует и завершает | выполняются захват штатно, зависаний интерфейса не выявлено 3 Захват трафика Появляется динамика | Телеметрия Успешно счетчиков и графиков | обновляется, поток данных обрабатывается 4 Детекция правил При тестовой Срабатывания Успешно активности фиксируются с формируются указанием типа и

события уровня 5 Конфигурируемость Изменение Изменения Успешно правил параметров влияет на | порогов/окон поведение детектора | применяются и отражаются в результатах 6 Журналирование Фиксируются Журнал содержит Успешно действия и записи по служебные операциям и сообщения режимам 7 Хранение в SQLite События Записи доступны Успешно сохраняются между после перезапуска сессиями приложения 8 Фильтрация событий Отбор по заданным Фильтры Успешно условиям работает применяются, корректно ВЫВОДЯТСЯ релевантные записи 9 Экспорт данных Формируются файлы | Экспорт Успешно выгрузки выполняется В предусмотренные форматы 10 | Базовая устойчивость Нет критических Критических сбоев | Успешно отказов В ТИПОВЫХ сценариях в ходе тестов не зафиксировано 39

Анализ таблицы 4.2 показывает, что прототип успешно проходит полный набор базовых проверок, предусмотренных планом испытаний. Это подтверждает достижение практической цели разработки в рамках проекта. 4.2.3 Оценка качества результата С инженерной точки зрения текущая версия HunterEye демонстрирует ряд значимых — сильных сторон, — подтверждающих — работоспособность и целесообразность выбранных — архитектурных решений. Совокупность полученных результатов свидетельствует о ТОМ, что заложенная в основу системы концепция оказалась технически состоятельной и способной служить надёжной основой для дальнейшего развития продукта: 1. функциональная полнота базового контура IDS, обеспечивающая целостную логическую цепочку от этапа захвата сетевого трафика до регистрации и визуального отображения инцидентов в интерфейсе; 2. эксплуатационная наглядность, благодаря которой пользователь может эффективно взаимодействовать с системой через графический интерфейс без необходимости обращения к исходному коду; 3. автономность развёртывания, позволяющая использовать систему в базовом сценарии без привлечения внешней инфраструктуры, такой как серверные базы данных или SIEM-платформы; 4. конфигурируемость детекции, обеспечивающая возможность гибкой настройки правил и параметров обнаружения через интерфейс или конфигурационные файлы; 5. воспроизводимость результатов, позволяющая повторять испытания по ключевым сценариям на аналогичных тестовых средах для проверки корректности работы системы. При этом важно учитывать ограничения текущей реализации, чтобы корректно определить границы применимости разработанного решения [32]. 4.2.4

Ограничения прототипа Несмотря На успешные результаты испытаний и — подтверждение работоспособности ключевых функций, разработанный прототип обладает рядом ограничений, характерных для учебно-прикладных систем обнаружения вторжений: 1. ограниченная наблюдаемость сети, при которой система анализирует только трафик, доступный выбранному сетевому интерфейсу, без поддержки распределённого сбора данных с нескольких узлов; 2. ограниченный набор правил, охватывающий лишь типовые сценарии атак и не включающий полный спектр современных и сложных угроз; 3. правило-ориентированный характер анализа, при котором поведенческие, эвристические и статистические методы обнаружения реализованы В минимальном объёме; 4. локальный масштаб хранения и — аналитики, — обусловленный использованием SQLite, что ограничивает возможности масштабирования, централизованной корреляции событий и работы в условиях высокой нагрузки; 40 5. базовый уровень автоматического реагирования, демонстрационном формате применения в промышленной среде. Для более наглядного представления ограничений и потенциальных направлений развития системы далее приведена сводная таблица. Таблица 4.3 реализованный В и требующий дальнейшего развития Для Ограничения текущей версии и направления развития

Ограничение Влияние на эксплуатацию | Возможное направление развития Анализ только Неполная картина сети в Добавление распределённых локально крупных сегментах агентов/сенсоров доступного трафика Ограниченный Часть угроз может не Расширение базы правил и сигнатур набор фиксироваться детекционных правил Преимущественно Вероятность ложных Введение адаптивных и ML- пороговая логика срабатываний/пропусков ПОДХОДОВ Локальное Ограничения Интеграция с серверной БД и SIEM хранилище SQLite масштабируемости и корреляции Базовые сценарии Ограниченная автоматизация | Развитие политик гевопве и

реагирования предотвращения сценариев оркестрации. Как видно из таблицы 4.3, проведённый анализ показывает, что большая часть выявленных ограничений относится не к ошибкам реализации, а к сознательно выбранному уровню сложности прототипа. В рамках проектной работы приоритет был отдан воспроизводимости, наглядности и модульности решения, что позволило реализовать устойчивый базовый контур IDS и обеспечить его проверку в практических сценариях. Такой подход является оправданным для учебно-прикладного проекта, где важна не максимальная инфраструктурная сложность, а демонстрация корректной инженерной логики построения системы. Отдельного внимания заслуживает вопрос масштабируемости. Текущая архитектура HunterEye ориентирована на локальный контур и может рассматриваться как базовый уровень для перехода к распределённой модели мониторинга. При — добавлении | удалённых | источников событий, централизованного хранилища и механизмов корреляции система способна эволюционировать в решение класса IDS/IDPS без отказа от уже реализованных модулей. С точки зрения качества реализации даже ограниченные компоненты прототипа обеспечивают полный цикл: сбор данных, анализ, фиксацию и отображение результатов. Это подтверждает корректность выбранной архитектуры и целесообразность дальнейшего развития без полной переработки системы. 41

Заключение В ходе выполнения проектной работы решена поставленная задача разработки прототипа системы обнаружения вторжений для локальной сети. По результатам предпроектного анализа были определены требования к системе, границы реализации и приоритетные сценарии обнаружения угроз. На основе выполненного обоснования технологий сформирован стек разработки, обеспечивший реализацию прикладного решения в формате настольного приложения. В рамках работы спроектирована и реализована модульная архитектура прототипа HunterEye, включающая подсистемы захвата трафика, детекции, журналирования, хранения данных, пользовательского интерфейса и базового реагирования. Реализованный конвейер обработки позволяет в режиме, близком к реальному времени, получать сетевые данные, применять правила обнаружения, формировать события безопасности и предоставлять результаты оператору в наглядной форме. Практическая реализация показала, что выбранный подход к построению IDS обеспечивает целостность и воспроизводимость работы системы в локальной среде Windows. Использование конфигурируемых параметров и интерфейсных средств управления позволило адаптировать поведение прототипа без внесения изменений в исходный код, что повысило удобство эксплуатации и сопровождения. По результатам тестирования подтверждена работоспособность ключевых функций: запуск и управление режимами работы, захват трафика, срабатывание детекционных правил, запись инцидентов в базу данных, фильтрация и экспорт событий, а также устойчивость в типовых пользовательских сценариях. Это позволяет считать цель дипломной работы достигнутой, а поставленные задачи — выполненными в полном объёме в рамках принятой постановки. Одновременно в работе зафиксированы ограничения текущей версии: локальный характер наблюдаемости — трафика, ограниченный набор реализованных правил, базовый уровень автоматического реагирования и отсутствие централизованной корреляции событий. Указанные ограничения соответствуют формату дипломного прототипа и задают направления дальнейшего развития. В качестве перспектив развития целесообразно выделить: расширение базы правил обнаружения, внедрение адаптивных методов анализа, интеграцию с внешними системами мониторинга и переход к распределённой модели сбора событий. Реализация этих направлений позволит повысить точность обнаружения, масштабируемость и практическую значимость системы. Таким образом, разработанный прототип HunterEye представляет собой завершённое учебно-прикладное решение, демонстрирующее основные принципы построения IDS для локальной сети и формирующее основу для дальнейших инженерных и исследовательских работ в области информационной безопасности. 42